

int は32ビット — 桁あふれを正しく折り返す

今日のゴール

`int` の演算を 32bit 命令 (`addw` など) で行い、桁あふれが C のとおりに折り返すようにする。

1 この回の位置づけ

- 意味論発展シリーズの第 2 回（選択制）。前提はコマ 10（型サイズ）
- S1・S3 とは独立している。3 本の中では最も「気づきにくい」ずれ
- `mycc.py` は書き換ええない。ラッパー `semcc.py`（完成品）が `emit` を通す
- 編集するのは `narrow.py` の 1 つの関数

2 いまのコンパイラは何をしているか

コマ 10 で「`int` は 4 バイト」と決め、`lw` / `sw` で読み書きするようにした。しかし計算のほうは、ずっと 64bit 命令のままである。

```
add a0, a1, a0      # 64bit で足している
```

RV64 には、同じ演算の 32bit 版がある。

64bit 版	32bit 版	32bit 版の意味
<code>add</code>	<code>addw</code>	下位 32bit で足し、結果を符号拡張する
<code>sub</code> / <code>mul</code> / <code>div</code> / <code>rem</code>	<code>subw</code> / <code>mulw</code> / <code>divw</code> / <code>remw</code>	同上
<code>sll</code> / <code>sra</code>	<code>sllw</code> / <code>sraw</code>	32bit としてシフトする

`w` は word（RV64 では 32bit）の意味である。`int` の計算には、本来こちらを使わなければならない。

3 何が違うのか

```
int x = 2147483647;    /* int の最大値 */
x = x + 1;
```

C の規則では、`int` は 32bit なので最小値 **-2147483648** に折り返す（厳密には符号付きオーバーフローは未定義動作だが、実際の処理系はほぼすべて折り返す。RISC-V もそう振る舞う）。

いまの実装は 64bit で計算するので、2147483648 という `int` に存在しないはずの値がそのまま残る。

```
int y = 100000 * 100000;    /* C: 1410065408 に折り返す */
                          /* いまの実装: 100000000000 */
```

重要

このずれも、S1 と同じくテストが通ってしまう種類のものである。講義のテストは小さい値しか使わないので、64bit でも 32bit でも同じ答えになる。大きな値を扱った瞬間に、初めて食い違う。

「小さい入力でテストする」のは正しい習慣だが、境界値を試さないと見つからないバグがあることも同時に示している。

4 どう直すか — 命令を書き換える

コード生成のあちこちに散らばった `add / sub / mul` … をすべて書き直すのは大変である。そこで、出力される命令行を通り道で書き換える。

```
codegen が emit("  add a0, a1, a0") を呼ぶ
↓
narrow(line, ty) が「型が int なら addw に」書き換える
↓
実際に出力される
```

`narrow` は純粋な文字列変換なので、テストが書きやすい。ラッパー `semcc.py` が `emit` を包んでこれと呼ぶ。

5 難しいのは「どこを書き換えないか」

素朴に「`add` を全部 `addw` にする」と、コンパイラは壊れる。

命令	用途	32bit にすると
<code>add a0, a1, a0</code>	<code>x + y</code> (int の足し算)	正しい
<code>add a0, a1, a0</code>	<code>p + i</code> (ポインタ演算)	アドレスが壊れる
<code>addi a0, s0, -24</code>	変数のアドレス計算	アドレスが壊れる
<code>addi sp, sp, -16</code>	スタックの確保	<code>sp</code> が壊れる

アドレスは **64bit** である。同じ `add` でも、「何を計算しているか」で扱いが変わる。

そこで 2 つの仕掛けを使う。

1. **`addi` は対象外にする** — アドレス計算専用と割り切る (このコンパイラは `int` の加算に `addi` を使わない)
2. **型コンテキストを見る** — いま生成している式の型が `int / char` のときだけ書き換える

2 つ目はラッパーが用意する。`codegen` を包んで、「いま生成中のノードの型」をスタックで持ち回り、`emit` のときに参照する。

補足

入れ子のアドレス計算に注意。`s = s + a[i]` を考える。外側の `+` は `int` だが、その中の `a[i]` はアドレス計算 (`base + i*4`) である。外側の型を内側にも適用してしまうと、アドレスが 32bit に切り詰められて落ちる。

ラッパーは、算術ノード以外では型を `None` にして積み直すことでこれを防いでいる。実際に作る途中でこの罠を踏み、`f11_array.c` が `SIGSEGV` で落ちた——「型は式ごとに違う」ことを、身をもって確かめられる箇所である。

6 実装

Step	実装対象	内容
1	<code>narrow(line, ty)</code>	型が <code>int</code> 系のときだけ、対象命令に <code>w</code> を付ける

書くのは 20 行ほどの文字列処理である。難しいのはコードではなく、何を対象にして何を対象にしないかの判断である。

```
python3 semcc.py tests/overflow.c | grep -E "addw|subw|mulw"
python3 semcc.py tests/pointer_ok.c | grep -c addiw    # 0 でなければ危険
```

7 テスト

```
python3 check.py      # narrow の入出力を直接確認 (未実装は SKIP)
python3 golden.py     # 折り返しのテスト + fixed15
```

テスト	確認内容
<code>overflow.c</code>	最大値 + 1 が最小値に折り返す
<code>mul_wrap.c</code>	<code>100000 * 100000</code> が 32bit で折り返す
<code>shift.c</code>	<code>1 << 31</code> が負になる、算術右シフト
<code>pointer_ok.c</code>	配列・ポインタ・構造体が壊れていない (<code>narrow</code> しすぎの検出)

`pointer_ok.c` が安全網である。`fixed15` と合わせて、「直しすぎていないこと」を保証する。

8 発展課題

1. **unsigned** を考える: `unsigned int` があれば `divu` / `remu` が要る。Core プロファイルに `unsigned` がないのはなぜか、あると何が增えるか

2. **long** との違い: C の **long** は 64bit である。仕様に **long** を足すとしたら、**narrow** はどう変わるか
3. **char** の扱い: **char** は 1 バイトだが、計算は **int** に格上げされる（整数拡張）。いまの実装は **char** も 32bit で計算しているが、これで正しいか
4. 比較は大丈夫か: **slt** は 64bit 比較だが、32bit で符号拡張された値同士なら正しく比較できる。なぜそう言えるか説明する
5. **B1** と組み合わせる: 定数畳み込み（B1）の **to_i64** は 64bit で丸めている。**int** の式なら 32bit で丸めるべきではないか。両者の整合を取る

補足

コラム: なぜ **RV64** に **32bit** 命令があるのか。RISC-V は 64bit CPU だが、C の **int** が 32bit であることを前提に、32bit 演算を 1 命令でできるようにしてある（**addw** など）。これがないと、足すたびに「下位 32bit を取り出して符号拡張する」処理が余分に要る。言語の都合が命令セットの設計に影響している例である。x86-64 も同じ理由で、32bit レジスタ名（**eax**）での演算が用意されている。